```
  def apply(): Behavior[Command] = ready()

  private def ready() = Behaviors.receiveMessage {
    case f: File =>
      ...                                          ❶
      busy()
  }

  private def busy() = Behaviors.receiveMessage {
    case File(location, camera) =>
      camera ! Camera.Photo(location)              ❷
      Behaviors.same
    case Done =>
      ready()
  }
}
```

❶ Processes the photo; when finished, returns to ready() by sending Done to itself

❷ Can’t process the file and sends back the photo to the camera

This plain actor can provide routing functionality by designating the next behavior to handle the next message—the most basic functionality of an actor.

In this chapter, you’ve learned how actors can find each other even if they are not in direct contact, thanks to the receptionist and built-in routers. With them, you’re one step closer to creating distributed and fault-tolerant systems with actors.

To be fail-safe even if the racks catch fire, you need to distribute the `ActorSystem`s across multiple machines. That is, you need clusters—and to build them, you first need to learn about Akka configuration in chapter 7.

Summary